Lab #1: Booting a RISC-V SoC with X-HEEP

March 26, 2026

1. Introduction

This lab introduces students to the **X-HEEP** platform, a lightweight and extensible RISC-V SoC template designed for **low-power edge computing applications**. The lab will cover setting up the development environment, running a simple "Hello World" application using simulation, exploring the main **X-HEEP hierarchy**, and understanding the role of **Makefiles, automation scripts, and FuseSoC** in managing the build process.

All the designs from this course will be made using **SystemVerilog**, a hardware description and hardware verification language standardized by the Institute of Electrical and Electronics Engineers (IEEE) as IEEE 1800-2023 [IEEE'24], which has become the de-facto standard in the open-hardware ecosystem. In particular, the code will be written according to the guidelines released by lowRISC.[1]

This teaching material has been prepared using information coming from the X-HEEP framework (see https://github.com/x-heep).

2. Set-up environment

The main architecture of the framework is depicted in Figure 1. The main elements are as following:

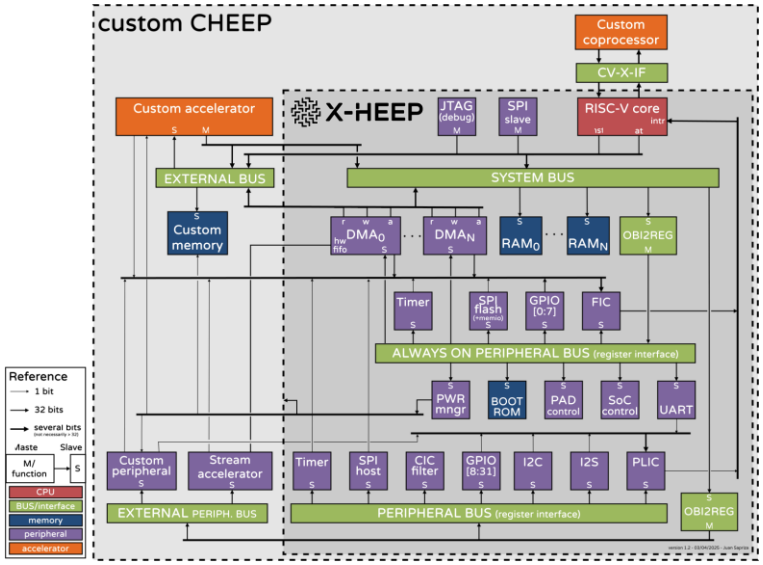


Figure 1. X-HEEP framework architecture

[1] https://github.com/lowRISC/style-guides/blob/master/VerilogCodingStyle.md

Today, we are going to focus on the X-HEEP part only. Thus, we will try now to set-up all the environment to be able to simulate the platform. Note that all the dependencies and initial tools are already installed into the PCs of the lab. They include everything described in https://x-heep.readthedocs.io/en/latest/GettingStarted/Setup.html.

- Open a terminal and clone the repository:

```
(base) jose@jose-PC:~/UPM/uCredentials$ git clone https://github.com/x-heep/GR-HEEP
Cloning into 'GR-HEEP'...
remote: Enumerating objects: 4464, done.
remote: Counting objects: 100% (940/940), done.
remote: Compressing objects: 100% (735/735), done.
remote: Total 4464 (delta 256), reused 547 (delta 164), pack-reused 3524 (from 1)
Receiving objects: 100% (4464/4464), 59.67 MiB | 34.23 MiB/s, done.
Resolving deltas: 100% (1268/1268), done.
(base) jose@jose-PC:~/UPM/uCredentials$
```

- Activate the conda environment (there should be a .sh file in this directory exporting ALL the needed paths):

  >> source /home/$USER/tools/x-heep.sh

- Open VSCode to see the whole framework

  >> cd GR-HEEP

  >> code .

At this point you are ready to start creating X-HEEP based SoCs and simulate them. Before that, let's try to understand different key elements.

3. Vendor, Main Makefile, automatization engines (mcu_gen.py) and Fusesoc

a) In X-HEEP we use the vendor tool from OpenTitan to manage the code which we include from external repositories. The vendor tool is a simple tool that allows you to copy upstream sources into the repository. You can find the vendor tool in ***util/vendor.py***, and the vendored repositories in *hw/vendor*.
In *hw/vendor* you will find a ***<organization>_<repo_name>.vendor.hjson*** file for each vendored repository. This vendor description file contains the information needed by the vendor tool to fetch the upstream repository and perform any necessary modifications to it.

b) The Makefile in X-HEEP is designed to streamline and automate the compilation, simulation, and FPGA synthesis processes. It provides a user-friendly interface for managing the entire SoC development flow, enabling easy execution of common tasks.
Key Features:
   - Application Compilation: Automatically compile RISC-V software applications.
   - Simulation Execution: Calls Verilator for cycle-accurate simulation.
   - FPGA Synthesis: Supports FPGA builds using Vivado.
   - Dependency Management: Ensures all required components are correctly built.

c) The **mcu_gen.py** script is an automation engine that dynamically configures the SoC memory map and peripheral interfaces. It simplifies hardware/software integration by: 1) Automatically generating HDL and C headers (for memory-mapped registers) from tpl files, 2) Ensuring consistency between hardware and software components, 3) Generating linker scripts based on the system's memory structure.

d) FuseSoC is an open-source package manager and build system for hardware design, primarily focusing on FPGA and ASIC development. It simplifies working with complex hardware projects by managing dependencies, configurations, and tool flows. It is the tool we use in X-HEEP to manage the different flows that we support. The main building blocks of FuseSoC are the ***.core files***. A *.core file* is self-contained description of a hardware

module, including its sources, dependencies, and configurations. You can find the .core files for X-HEEP all around the project, especially in the hw directory and its subdirectories. However, the main .core file is core-v-mini-mcu.core. To understand .core files, modify them, or write your own, please check the documentation on this in the FuseSoC documentation. X-HEEP's Makefile uses FuseSoC behind the scenes to build, simulate, and synthesize the design, among other uses.

After knowing these tools, we can issue the following command

>> make mcu-gen

```
make verible
make[3]: Entering directory '/home/jose/UPM/uCredentials/GR-HEEP/hw/vendor/x-heep'
$You are executing from: /home/jose/UPM/uCredentials/GR-HEEP/hw/vendor/x-heep
USING MINICONDA core-v-mini-mcu
util/format-verible;
make[3]: Leaving directory '/home/jose/UPM/uCredentials/GR-HEEP/hw/vendor/x-heep'
make[2]: Leaving directory '/home/jose/UPM/uCredentials/GR-HEEP/hw/vendor/x-heep'
make[1]: Leaving directory '/home/jose/UPM/uCredentials/GR-HEEP'
make verible
make[1]: Entering directory '/home/jose/UPM/uCredentials/GR-HEEP'
USING MINICONDA core-v-mini-mcu
hw/gr-heep/gr_heep_bus.sv:139:9-22: Do not use $test$plusargs to access plusargs, use $valu
e$plusargs instead. [Style: plusarg-value-assignment] [plusarg-assignment]
make[1]: Leaving directory '/home/jose/UPM/uCredentials/GR-HEEP'
✅ DONE! X-HEEP MCU and GR-HEEP generated successfully
(core-v-mini-mcu) jose@jose-PC:~/UPM/uCredentials/GR-HEEP$
```

This will generate the basic architecture consisting on 2 memory banks of 32KB and the cv32e20 with the configuration stated in:

```
EXPLORER
GR-HEEP
  .vscode
  config
    gr-heep_pad_cfg.py
    mcu-gen-config.hjson
    mcu-gen-config.py
  hw
  scripts
  sw
  tb
  util
    vendor.py
  .gitignore
  gr-heep.core
  LICENSE
  Makefile
  README.md
```

```
mcu-gen-config.py 9
config > mcu-gen-config.py > ...
42
43  def config():
44      # Parallel bus
45      system = XHeep(BusType.NtoM)
46
47      # Set cv32e40px CPU
48      system.set_cpu(CPU("cv32e40px"))
49
50      # Memory subsystem
51      # - 2 x 32kiB firmware and data
52      # - 2 x 16kiB interleaved banks
53      memory_ss = MemorySS()
54      memory_ss.add_ram_banks([32] * 2)
55      memory_ss.add_ram_banks_il(4, 16, "data_interleaved")
56      # Linker script sections
57      memory_ss.add_linker_section(LinkerSection.by_size("code", 0, 0x0000E800))
58      memory_ss.add_linker_section(LinkerSection("data", 0x0000E800, None))
59      system.set_memory_ss(memory_ss)
60
```

The easiest way of changing the default values is with some arguments. For example, you can change the CPU type (cv32e20), the default bus type (NtoM), the default continuous memory size (2 banks), or the default interleaved memory size (4 banks) with

>> make mcu-gen CPU=cv32e40p BUS=NtoM MEMORY_BANKS=12 MEMORY_BANKS_IL=4

```
171  # Export variables
172  export HEEP_DIR = $(X_HEEP_DIR)
173  export X_HEEP_CFG
174  export PADS_CFG
175  export EXTERNAL_DOMAINS
176  export XHEEP_CONFIG_CACHE
177  export PROJECT
178
```

This generates X-HEEP with the cv32e40p core, a parallel bus, and 16 memory banks (12 continuous and 4 interleaved), 32KB each, for a total memory of 512KB.

Note that after performing the *make mcu-gen …* command(s), the different .sv file from the tpl.sv files are created as well as the headers in the HAL of the peripherals containing the memory mapped structures (to easily access in the code to the different registers or bitfields).

4. Simulating "hello world" App with Verilator and checking signals with GTKWave

Verilator is an open-source Verilog and SystemVerilog simulator that compiles into multithreaded C++ or SystemC code.[2] It is well-known and has been widely adopted by both community and industry, as many open-source hardware projects rely on it as their main simulation infrastructure.

After having generated the system with the required amount of memory, it is time to simulate using Verilator.

>> make verilator-build

This will create the build folder in root, where the model has been compiled. Then, we can just run the app on it:

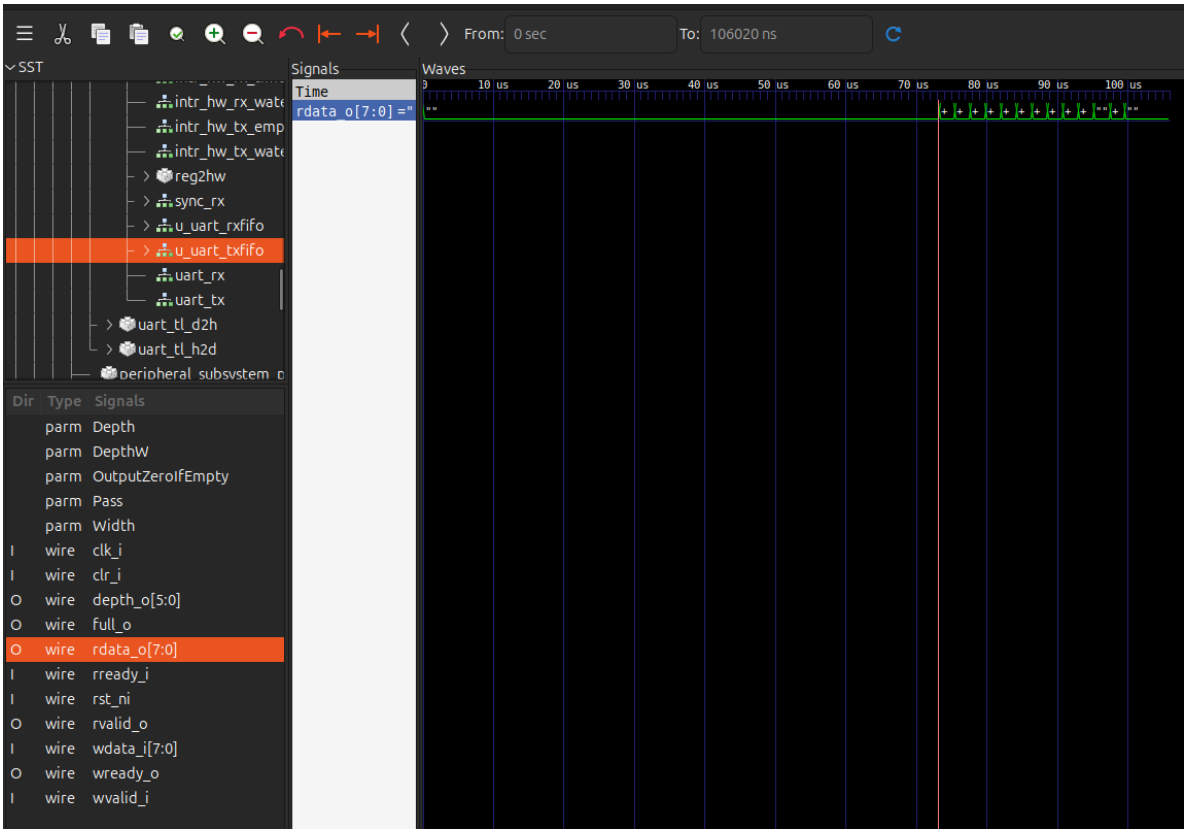
>> make verilator-run-app

```
get=sim --tool=verilator  \
        --run x-heep:systems:gr-heep  \
        --run_options="+firmware=../../../sw/build/main.hex "
WARNING: Unknown item cache_type in section Generators
WARNING: Unknown item file_input_parameters in section Generators
WARNING: Unknown item include_dirs in section Fileset
WARNING: Unknown item formal in section Target
WARNING: Unknown item formal in section Target
WARNING: Unknown item formal in section Target
INFO: Running
INFO: Running pre_run script remove_uart_log
INFO: Running simulation
[TESTBENCH]: No OpenOCD is used
[TESTBENCH]: loading firmware  ../../../sw/build/main.hex
[TESTBENCH]: No Max time specified
[TESTBENCH]: No Boot Option specified, using jtag (boot_sel=0)
[X-HEEP]: NUM_BYTES =        128KB

UART: Created /dev/pts/4 for uart0. Connect to it with any terminal program, e.g.
$ screen /dev/pts/4
UART: Additionally writing all UART output to 'uart0.log'.
Reset Released
Set Exit Loop
Memory Loaded
Simulation finished after 10602 clock cycles
Program Finished with value 0
INFO: Running post_run script print_uart_log
hello world!
```

[2] https://www.veripool.org/verilator/

After the simulation, you can inspect the signals using GTKWave, open with GTKWave the waveform.vcd file located into the sim-verilator folder in the build folder. Then look for testharness and for the data fifo of the UART to see the “hello world!” print.



4.1. Playing with the configs.

To play and see the effect of the different possible configurations, please execute the following commands:

>>make mcu-gen PYTHON_X_HEEP_CFG=/home/.../GR-HEEP/config/e20_one2M_10B.py X_HEEP_CFG=/home/.../GR-HEEP/hw/vendor/x-heep/configs/python_unsupported.hjson

After having generated the system with the required amount of memory, it is time to simulate using Verilator.

>> make verilator-build

This will create the build folder in root, where the model has been compiled. Then, we can just run the app on it:

>> make verilator-run

Note that there could be different configurations with different results (cycles)

Config	CPU	Bus	Memory	DMA setup	Main intent
e20_one2M_10B.py	CV32E20	ONE2M	NORMAL	1 channel	Baseline reference point

e40p_N2M_2B_INT8B.py	CV32E40P	N2M	INTL	1 channel	Better core + more parallel memory/interconnect behavior
e40p_one2M_10B.py	CV32E40P	ONE2M	NORMAL	1 channel	Isolate core effect while reverting interconnect/memory
e40p_xpulp_N2M_2B_INT8B.py	CV32E40P + XPULP	N2M	INTL	1 channel	Add ISA acceleration on top of a high-throughput fabric

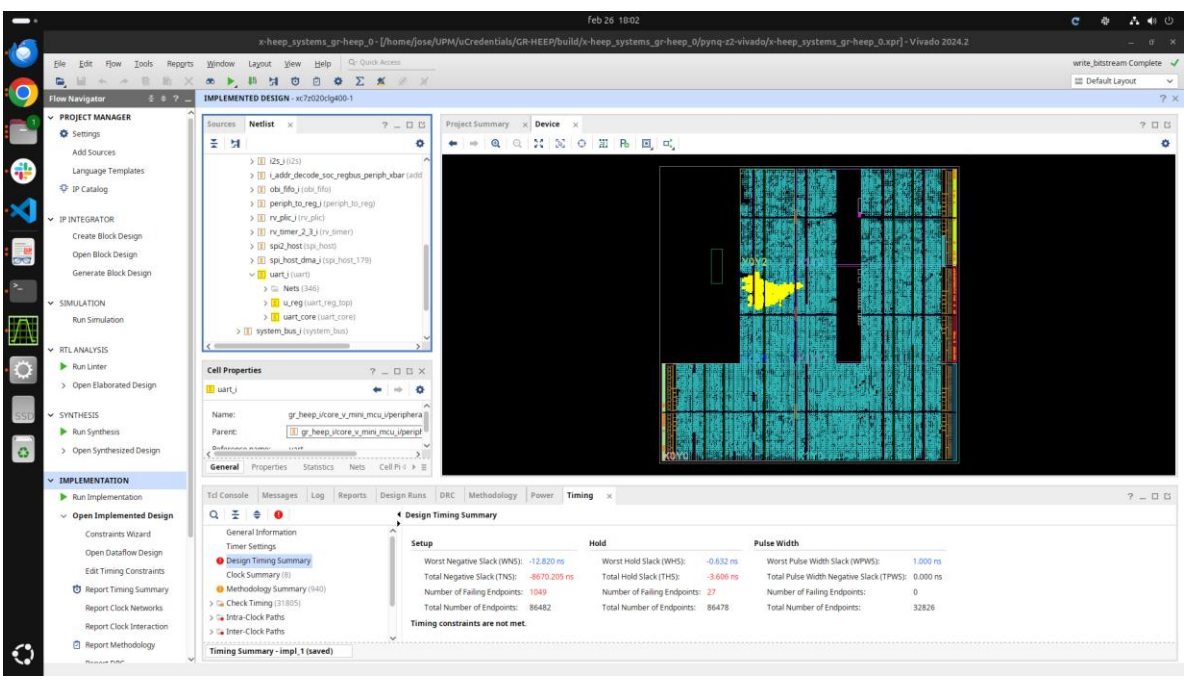
5. Vivado and main X-HEEP hierarchy

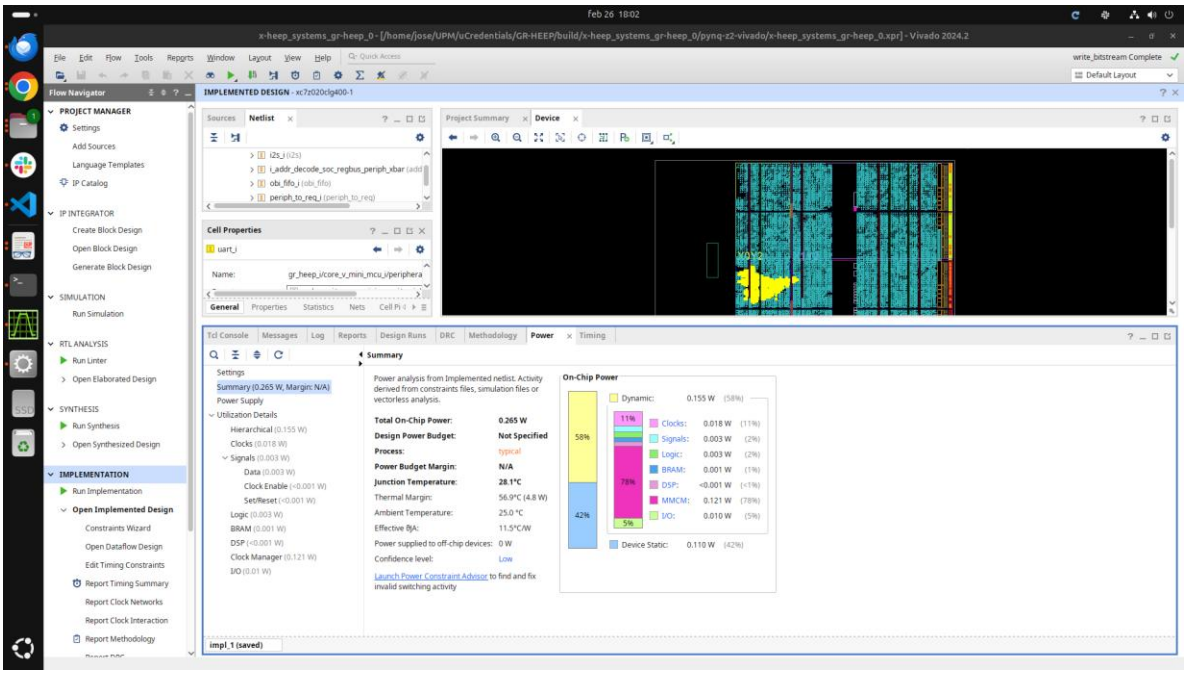
To implement the system and generate the bitstream to load into the FPGA, type:

>> make vivado-fpga FPGA_BOARD=pynq-z2

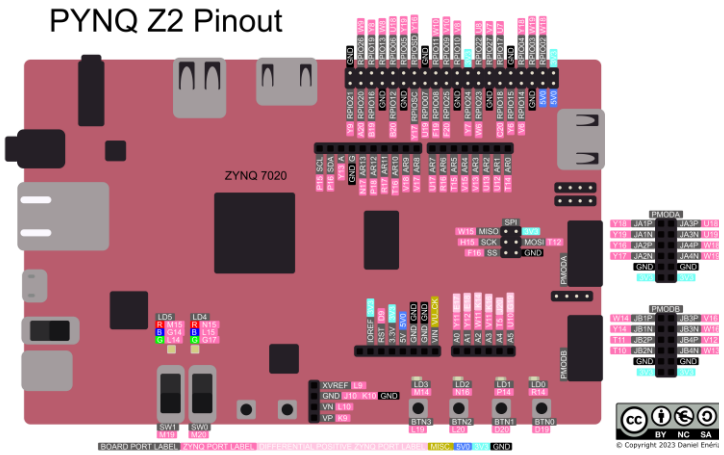
This would configure and create a vivado project and a bitstream to prototype into the pynq-z2 board. Note that there are more alternatives, such as nexys and zcu variants. In case you do not have capabilities to run the implementation with vivado, please download an already implemented project **here**.

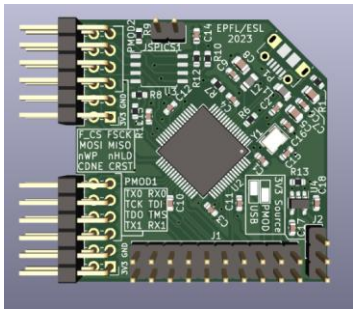
After that, open vivado and explore the hierarchy of the system as well as the estimated power consumption when prototyping into the FPGA:





To implement into the FPGA you can issue the corresponding “vivado-fpga-pgm" command or just open the HW Manager of vivado, select the generated bitstream and program the FPGA. After that, connect the programmer to the FPGA and you will be able to issue the following commands. Before that set properly SW1 and SW0:





PYNQ Z2 Pinout v2. Correction of Arduino header SDL and SDA pins and Raspberry Pi header RPIO06 pin.

boot_sel_i	execute_from_flash_i	boot procedure
0	X	JTAG
1	1	SPI Flash Execution
1	0	SPI Flash Loading

SW0 is boot_sel_i, and SW1 is execute_from_flash_i --> Set it to flash load mode.

>> make flash-readid

```
yosyshq_icestorm/iceprog'
init..
cdone: high
reset..
cdone: high
flash ID: 0xEF 0x70 0x18 0x00
cdone: high
Bye.
make[1]: Leaving directory '/home/jose/UPM/uCredentials/GR-HEEP/hw/vendor/x-heep'
(core-v-mini-mcu) jose@jose-PC:~/UPM/uCredentials/GR-HEEP$
```

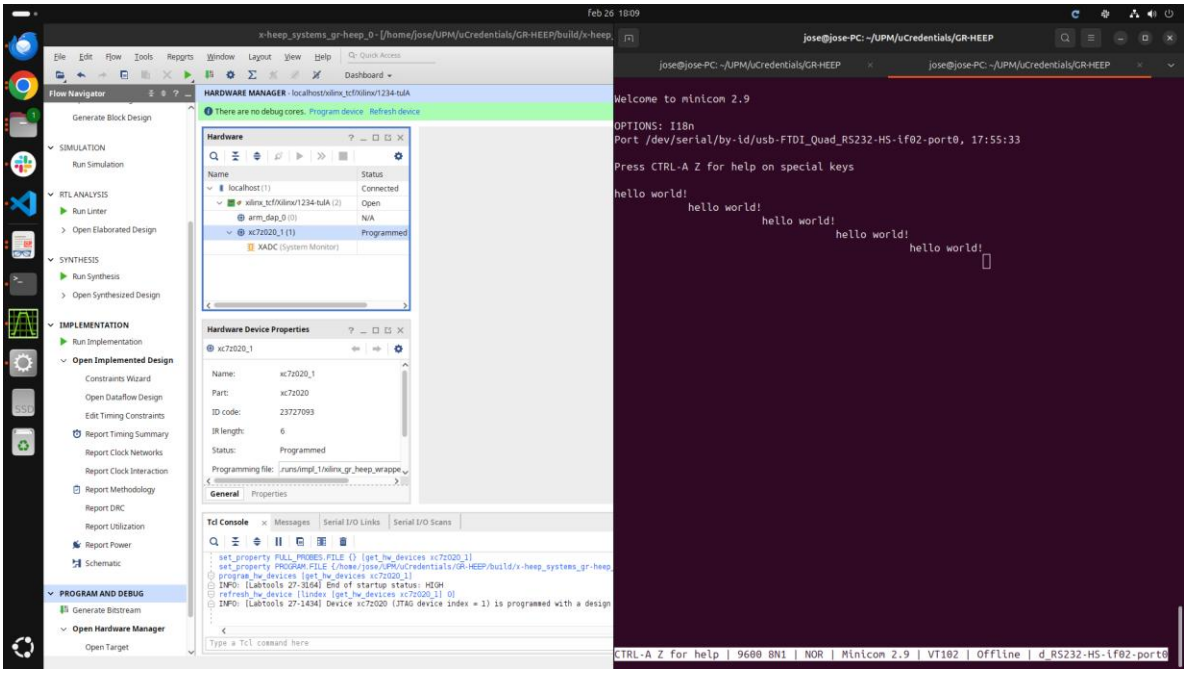
>> make app TARGET=pynq-z2 LINKER=flash_load

```
make[5]: Leaving directory '/home/jose/UPM/uCredentials/GR-HEEP/hw/vendor/x-heep/sw/build'
[100%] Built target main.elf
make[4]: Leaving directory '/home/jose/UPM/uCredentials/GR-HEEP/hw/vendor/x-heep/sw/build'
make[3]: Leaving directory '/home/jose/UPM/uCredentials/GR-HEEP/hw/vendor/x-heep/sw/build'
make[2]: Leaving directory '/home/jose/UPM/uCredentials/GR-HEEP/hw/vendor/x-heep/sw'
Total space: 128.0 kB = Continuous: [32, 32] kB + Interleaved: [16, 16, 16, 16] kB
Region  Start   End     Sz(kB)  Usd(kB) Req(kB) Utilz(%)
Code:     0.0    58.0    58.0     8.4     8.4    14.6
Data:    58.0    64.0     6.0     5.0     5.0    83.6
ILdata:  64.0   128.0    64.0     0.0     0.0     0.0

Cont 0 CCCCCCCCC----------------------          26.4%
Cont 1 -------------------------dddddd          15.7%
IntL 2 ----------------           0.0%
IntL 3 ----------------           0.0%
IntL 4 ----------------           0.0%
IntL 5 ----------------           0.0%
make[1]: Leaving directory '/home/jose/UPM/uCredentials/GR-HEEP/hw/vendor/x-heep'
(core-v-mini-mcu) jose@jose-PC:~/UPM/uCredentials/GR-HEEP$
```

>> make flash-prog

```
init..
cdone: high
reset..
cdone: high
Disabling Quad operation...
SR2: 00000000
flash ID: 0xEF 0x70 0x18 0x00
provided file size: 59712
erase 64kB sector at 0x000000..
programming..
done.
reading..
VERIFY OK
cdone: high
Bye.
make[1]: Leaving directory '/home/jose/UPM/uCredentials/GR-HEEP/hw/vendor/x-heep'
(core-v-mini-mcu) jose@jose-PC:~/UPM/uCredentials/GR-HEEP$
```

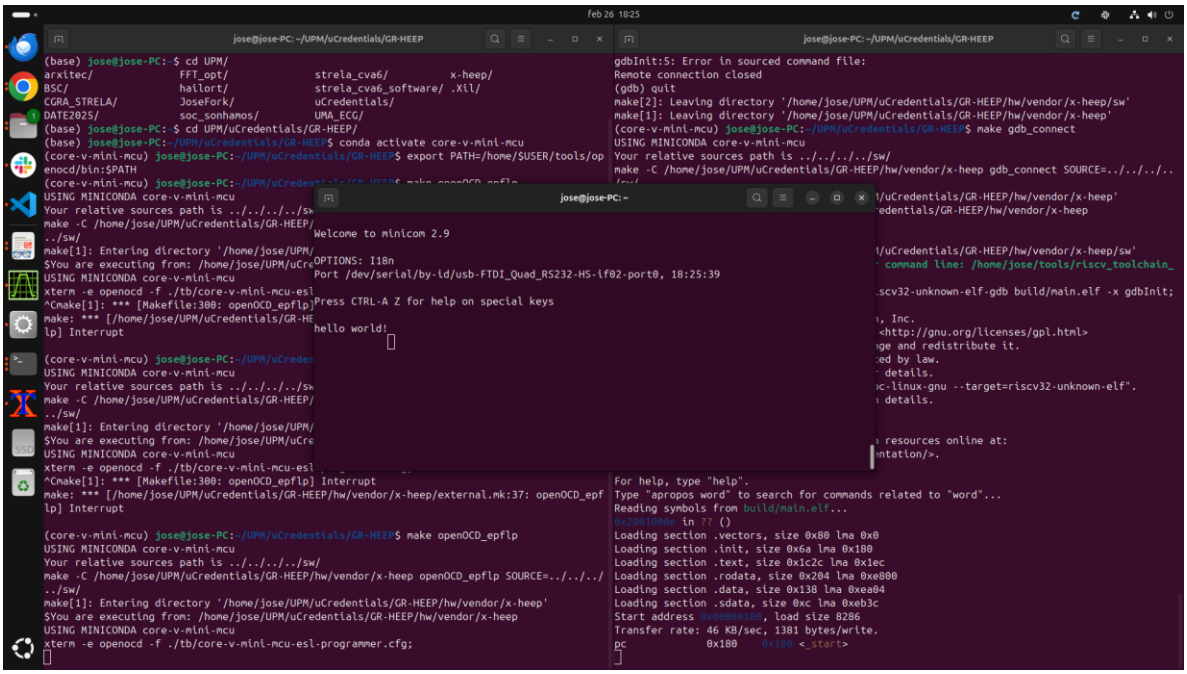


Another option is to change the SWs for on-chip or JTAG mode, and then execute:

>> make openOCD_epfl

>> make gdb_connect

>> minicom -b 9600 -D /dev/serial/by-id/usb-FTDI_Quad_RS232-HS-if02-port0



Additionally, you have as well this script that you could use to automate all the programming process (change line picocom by minicom command):

https://github.com/JoseCalero/x-heep/blob/main/util/flash_script.sh

References

[IEEE'24] IEEE, "IEEE Standard for SystemVerilog–Unified Hardware Design, Specification, and Verification Language", in *IEEE Std 1800-2023 (Revision of IEEE Std 1800-2017)*, pp. 1–1354, 2024.

Lab #2: ADDING A SIMPLE COUNTER PERIPHERAL THROUGH THE XAIF

Prerequisites

This tutorial assumes that you are familiar with the base X-HEEP and have set up and can simulate the execution of basic applications using verilator. For an introduction, follow the instructions available in X-HEEP's documentation[3].

Introduction to the XAIF

The open-source[4] X-HEEP platform [1] allows to plug in accelerators natively to the system bus through the eXtendible Accelerator InterFace (XAIF). The bus exposes a configurable number of slave and master ports to the external XAIF interface so that one or more accelerators with different bandwidth constraints can be connected.

As can be seen in the top left of Figure 1, the XAIF provides enhanced connectivity to external custom accelerators and includes the following ports:

- Slave and master ports: A configurable number of master and slave interfaces based on the OBI bus protocol[5] or on the register interface[6] and connected to the X-HEEP internal system bus, enabling seamless integration of custom-designed accelerators or peripherals.

[3] https://x-heep.readthedocs.io

[4] https://github.com/x-heep/x-heep

[5] You can read the specifications in https://github.com/openhwgroup/obi.

[6] You can find more information in https://github.com/pulp-platform/register_interface

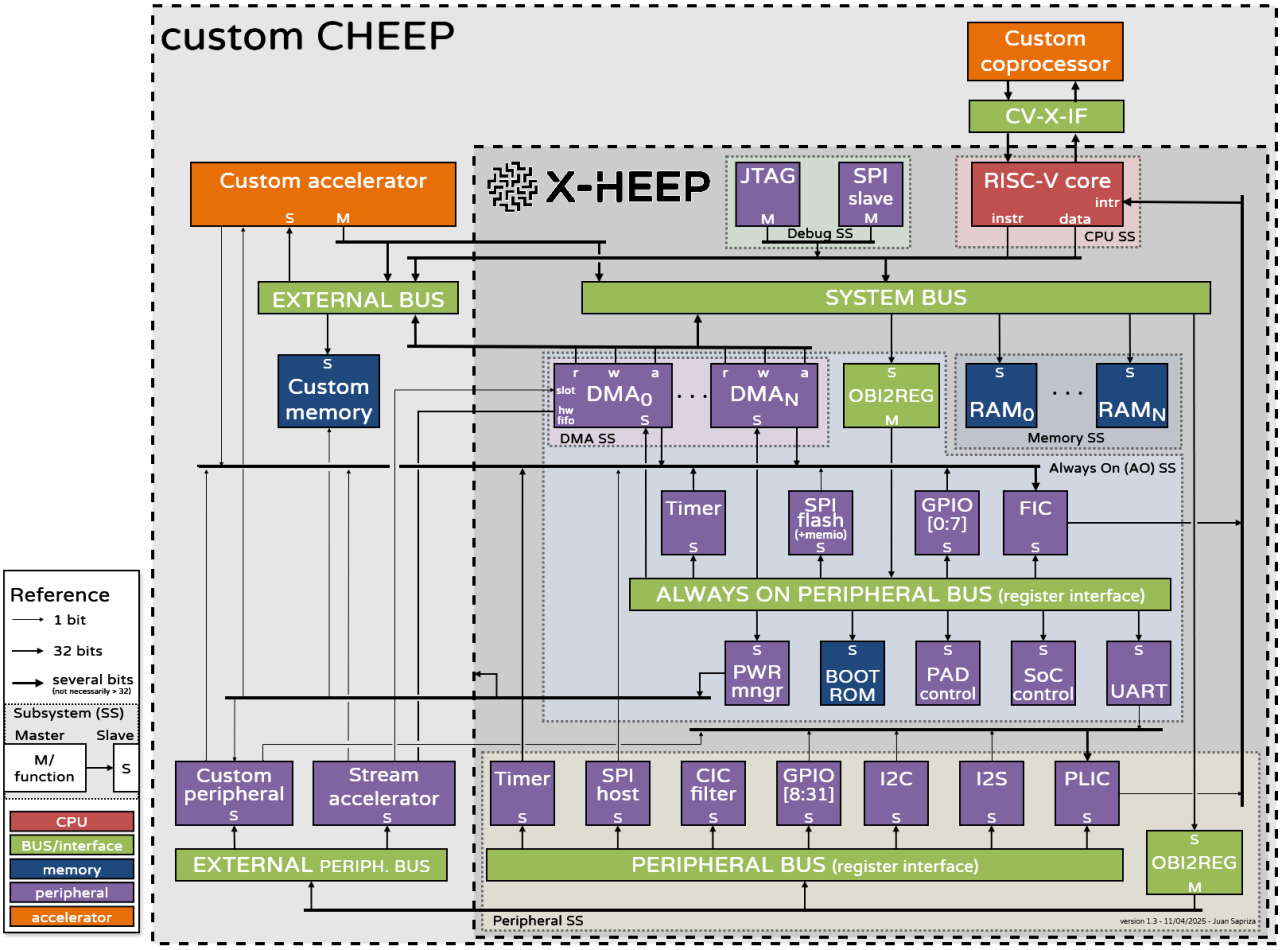


Figure 1: Diagram of X-HEEP. The eXtendible Accelerator InterFace (XAIF) is shown in the top left corner as the connections to the custom accelerator. The CV-X-IF coprocessor and accelerator shown in orange are your custom modules to extend X-HEEP.

- Interrupt ports: A configurable set of interrupt lines linked to the X-HEEP internal PLIC interrupt controller, allowing accelerators to notify the host CPU upon operation completion.

- Power control ports: A configurable number of power control interfaces connected to the X-HEEP internal power manager, providing low-power management capabilities for connected accelerators.

Introduction to the Simple Counter Peripheral

In this tutorial, we are going to integrate a simple software-controlled up counter[7] as a peripheral to X-HEEP. The counter implements the OBI bus interface for the counter value and the register interface for the configuration registers. It is controlled using the control and status registers documented in the repository. This simple example will showcase the steps needed to connect more complex accelerators.

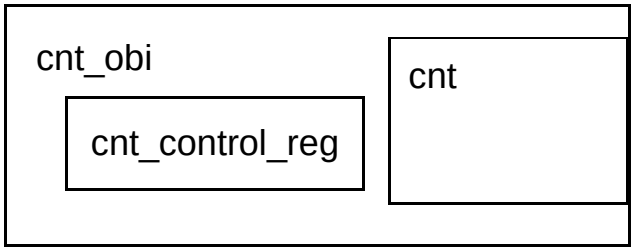


Figure 2: Top-level diagram of the simple counter.

The top-level module for the counter is in hw/cnt.sv, and its OBI bus wrapper is in hw/cnt_obi.sv. This wrapper also contains the control registers (Figure 2). Use a few minutes to get familiar with these modules, especially cnt_obi, as you will later have to connect this to X-HEEP.

eXtending X-HEEP

Until now, we have two different repositories: X-HEEP and the simple counter. Now is the time to put them in the same place. For this, we will have to create a new top-level repository that englobes both. If you already have a repository extending X-HEEP, you can reuse the same one instead of creating a new top level. In this case, skip the next paragraphs until making sure that everything is set up properly and start adding the simple counter.

The procedure of extending X-HEEP is detailed in its documentation[8]. However, since the base X-HEEP platform is meant to be extended, as Figure 1 shows, the open-source[9] GRHEEP repository already provides a top-level baseline that we can use.

To start, we will use a specific GR-HEEP commit (575cdd8) as a template to create your own top-level repository. This will ensure the reproducibility of this tutorial. To do this, you can run:

```
git clone git@github.com:x-heep/GR-HEEP.git

cd GR-HEEP

git checkout 575cdd8
```

At this point, make sure that everything is set up properly by simulating the hello world app using Verilator in GR-HEEP, following the same commands as in X-HEEP. Once you have your top-level repository, you have to add the counter to it. For this, X-HEEP uses the Vendor tool from OpenTitan[10]. Vendor can import hardware modules and primitives that we want to use inside our project. For more information,

[7] https://github.com/x-heep/simple_cnt
[8] https://x-heep.readthedocs.io/en/latest/Extending/index.html
[9] https://github.com/x-heep/GR-HEEP
[10] https://opentitan.org/book/util/doc/vendor.html

you can check the X-HEEP docs[11]. To know where to fetch the files from, Vendor needs a .vendor.hjson file like the one you can find for X-HEEP in hw/vendor/x-heep.vendor.hjson. Add your own vendor file for the simple_cnt based on the one for X-HEEP. Again, for reproducibility, we will use a specific commit (186e1f4). You can ignore and delete the patch_dir and the exclude_from_upstream sections. Then, after remembering to activate X-HEEP's conda environment, from the top-level directory run:

```
make vendor-update MODULE_NAME=simple_cnt
```

You should see the simple_cnt project copied into the hw/vendor directory. Now, in the configuration files, we have to add the information on the simple counter. First, add it to the config/mcu-gen-config.py under the slaves memory map in gr_heep_config(). The default offset and length are more than enough.

```
ext_xbar_slaves = {
        "simple_cnt": {
                "offset": 0x00000000,
                "length": 0x00010000,
        }
}
```

Then, add it to the external peripherals.

```
ext_periph = {
        "simple_cnt": {
                "offset": 0x00000000,
                "length": 0x00001000,
        }
}
```

Finally, set **the external_interrupts to 1.** The next step is instantiating the simple counter module as an external peripheral. This is done in the hw/gr-heep/gr_heep_peripherals.sv.tpl[12]. You can instantiate it and connect it to the proper signals like this:

```
% for a_slave in gr_heep["peripherals"]:
        % if (a_slave['name'] == "SimpleCnt"):
            cnt_obi #(
                    .W (32)
            ) u_cnt_obi (
                    .clk_i(clk_i),
                    .rst_ni(rst_ni),
                    .reg_req_i(gr_heep_peripheral_req[gr_heep_pkg::SimpleCntPeriphIdx]),
                    .reg_rsp_o(gr_heep_peripheral_rsp[gr_heep_pkg::SimpleCntPeriphIdx]),
                    .obi_req_i(gr_heep_slave_req_i[gr_heep_pkg::SimpleCntIdx]),
                    .obi_rsp_o(gr_heep_slave_resp_o[gr_heep_pkg::SimpleCntIdx]),
                    .tc_int_o(gr_heep_peripheral_vec_int[0])
            );
        % endif
% endfor
```

[11] https://x-heep.readthedocs.io/en/latest/GettingStarted/UnderstandingTools.html

[12] To learn more about .tpl files, you can check the configuration documentation for X-HEEP: https://x-heep.readthedocs.io/en/latest/Configuration/index.html

FuseSoC[13] is used in X-HEEP to create a list of hardware modules and interface with electronic design automation (EDA) tools like verilator, questasim, or vivado. To add the simple counter to the project's dependencies, you have to modify the gr-heep.core file. In the dependencies of the files_rtl_generic fileset, add a new line with the name that is defined in the hw/vendor/simple_cnt/cnt.core file (i.e. polito:isa-lab:cnt)[14].

At this point, the connection of the simple counter in hardware should be complete. Verify this by running the hello world with the updated files and fix any errors. Remember to generate the updated files from the templates and configurations with make mcu-gen.

Interacting With and Testing the Simple Counter

Now that the hardware is in place, we have to add the software drivers to interact with the simple counter. For this, you must add three symbolic links in sw/external/lib/ drivers/simple-cnt folder to:

- hw/vendor/simple_cnt/sw/cnt_control_reg.h. This will include some definitions for the control registers of the counter.
- hw/vendor/simple_cnt/sw/simple_cnt.h, which is the header file of the driver.
- hw/vendor/simple_cnt/sw/simple_cnt.c, which is the actual implementation of the driver.

Finally, it is time to write an application that will make use of the hardware counter. For that, you have to create a new folder in sw/applications with the name of your application and add a main.c. You can find an example in Appendix A.1.

If needed, increase the number of memory banks of the system by modifying the memory_ss in config/mcu-gen-config.py and re-run make mcu-gen.

```
memory_ss.add_ram_banks([32] * 4)
```

Verify that the simple counter is working properly by running the application and checking the waveforms. Remember, you can check what targets are available in the top-level Makefile by opening it or running make help. You can view the simulation run with Verilator using GTKWave. You will find the generated .fst file in build/.../sim-verilator/ waveform.fst. The design will be in TOP.testharness.gr_heep_i.

Next Steps

Think of some modifications to the simple counter example that would accelerate a simple application and try to implement them. You can start from hw/vendor/simple_ cnt/hw/cnt.sv. Since this is a demo, you can directly modify the RTL of the counter in hw/vendor/simple_cnt. However, keep in mind that this is not the intended way of using the vendor tool. The correct flow in a collaborative environment would be to update the simple_cnt repository, update which revision (commit) we want to point to in

[13] https://fusesoc.readthedocs.io

[14] For more details, you can check X-HEEP's documentation on FuseSoC and .core files: https:// x-heep.readthedocs.io/en/latest/GettingStarted/UnderstandingTools.html, or FuseSoC's user guide: https://fusesoc.readthedocs.io/en/stable/user/index.html

hw/vendor/simple_cnt.vendor.hjson, and run again the vendor-update make command to "pull" the changes.

Test your modifications to the counter by updating the application that uses it. You can also play around with the application. Remember that you can also modify the drivers in sw/external/lib/drivers/simple_cnt.

References

[1] S. Machetti, P. D. Schiavone, T. C. Müller, M. Peón-Quirós, and D. Atienza, "X-HEEP: An Open-Source, Configurable and Extendible RISC-V Microcontroller for the Exploration of Ultra-Low-Power Edge Accelerators," Jan. 2024.

Appendix A Simple Counter Application

A.1 main.c

```
// System library headers
#include <stdio.h>

// Custom library headers
#include "core_v_mini_mcu.h"
#include "fast_intr_ctrl.h"
#include "simple_cnt.h"
#include "ext_irq.h"
#include "csr.h"

// Main body
int main(void)
{
    uint32_t val = 255;
    uint32_t ret = 0;

    // Initialize external interrupts handler(s)
    ext_irq_init();

    // Set the counter threshold
    simple_cnt_set_threshold(val);

    // Read back the counter threshold
    ret = simple_cnt_get_threshold();
```

```
    if (val != ret) {
        printf("Error: threshold value mismatch\n");
        return -1;
    }

    // Set the counter value
    val = 42;
    simple_cnt_set_value(val);

    // Read back the counter value
    ret = simple_cnt_get_value();
    if (val != ret) {
        printf("Error: value mismatch\n");
        return -1;
    }

    // Clear the counter
    simple_cnt_clear();

    // Read back the counter value
    ret = simple_cnt_get_value();
    if (0 != ret) {
        printf("Error: value mismatch\n");
        return -1;
    }

    // Enable the counter
    simple_cnt_enable();

    // Wait for the counter to reach the threshold
    simple_cnt_wait_poll();
    printf("TC set\n");

    // Disable the counter and clear it
    simple_cnt_disable();
    simple_cnt_clear();

    // Install the counter interrupt handler and enable the counter
    simple_cnt_irq_install();  // install the handler in X-HEEP's PLIC
    simple_cnt_enable();       // start counting
```

```
    // Wait for the counter interrupt
    simple_cnt_wait_irq();     // put CPU to sleep until interrupt is
received
    simple_cnt_disable();      // disable counter to avoid further
interrupts
    printf("IRQ received\n");

    // Read back the counter value
    ret = simple_cnt_get_value();
    printf("Done. Counter value: %d\n", ret);

    return 0;
}
```